Where Should Resident Layers Go? Optimal Placement for Weight Streaming in Offloaded LLM Inference

Abstract

Models that exceed GPU memory can be served by keeping some layers resident and streaming the others from host-attached memory, such as page-locked DRAM or a CXL expander, through a ring of GPU buffers. Where should the resident layers sit? We model the pipeline as a max-plus linear system whose iteration time is the maximum mean of a family of tilings of the layer sequence. For L layers of equal compute time c and copy time f, r resident, m streamed and $k \le m$ buffers we prove three results: resident layers are useless when a layer computes at least as long as it copies; otherwise, spreading the resident layers evenly attains the lower bound $\max(Lc, mf)$ whenever $(k-1)f \ge 2c$, so three buffers suffice; and a contiguous block of resident layers, as produced by model-order filling, takes $\max(Lc,\ mf,\ rc + 2c + (m-k)f)$, up to $2 - c/f$ times the optimum. On a Tesla T4 with page-locked copies at 12.3 GB/s, similar to current CXL expanders, real Llama-3.2-3B and Llama-3.1-8B layer kernels and per-layer times measured on that GPU, the model reproduces 605 measured iteration times with a mean error of 0.29% and orders 98.6% of distinguishable placement pairs correctly. Even spreading is up to 1.50× faster than a contiguous block and reaches the bound with two buffers where the block needs five. Because the T4 is power-limited, the main measurements lock its clock.

Keywords: LLM inference, weight offloading, prefetching, max-plus algebra, layer placement, host-attached memory, Compute Express Link, GPU

1. Introduction

A large language model whose weights do not fit in GPU memory can still be served on one GPU by keeping part of the weights in host-attached memory and copying the remaining layers into GPU memory while earlier layers compute [1–3]. The host-attached tier is usually page-locked DRAM; Compute Express Link (CXL) memory expanders extend it with capacity that the host maps as ordinary memory, and that a GPU reaches through the same host-initiated DMA path [4, 5]. On this path the transfer bandwidth is one to two orders of magnitude below GPU memory bandwidth (about 12 GB/s for PCIe Gen3 ×16 and of the same order for current CXL prototypes [4–6]), so the copies are the bottleneck unless they overlap with computation.

Every such runtime makes two decisions that are rarely discussed separately: how many GPU buffers (*slots*) to reserve for layers in transit, and *which* layers to keep permanently resident in the remaining memory. The second decision has an obvious part, how many layers fit, and a less obvious part, *where* in the layer sequence the resident layers sit. Popular tools fill the GPU in model order: the Hugging Face Accelerate `device_map` assigns modules to the GPU until it is full and offloads the rest [3], which makes the resident layers a contiguous prefix of the model. FlexGen keeps the same fraction of every layer's weights on the GPU [1], and an edge-serving system spreads retained layers evenly over its pipeline stages [7]. None of these choices is justified by an analysis of the transfer pipeline, and we did not find a study that quantifies how much the position of the resident layers matters.

This paper gives that analysis and checks it on a GPU. We model one inference iteration as a cyclic sequence of layer units, a single copy engine and a ring of k slots, and we show that the steady-state iteration time is the maximum circuit mean of a max-plus linear system, which we characterise exactly. The model yields closed forms and a small set of rules:

1. **Exact iteration time** (Section 3). For any per-layer compute and transfer times, any resident set and any number of slots, the iteration time is the maximum mean weight of a family of *tilings* of the layer sequence. The recurrences evaluate it by simulation in time linear in the number of simulated layer executions, and with per-layer times measured on the same GPU it reproduces the measured iteration time with a mean error of 0.29% (Section 6.1).
2. **Optimal placement for uniform layers** (Section 4). If a layer computes at least as long as it takes to copy, any placement with two slots runs at the speed of resident weights, and resident layers buy nothing. Otherwise, spreading the resident layers evenly over the model attains the lower bound $\max(Lc, mf)$ whenever $(k-1)f \ge 2c$, so three slots suffice whenever at least three layers are streamed, and two suffice when a copy takes at least twice as long as a layer's computation.
3. **The cost of contiguous placement** (Section 4). For a contiguous resident block we derive the exact iteration time $\max(Lc,\ mf,\ rc + 2c + (m-k)f)$. It exceeds the optimum by a factor below $2 - c/f$, a bound that is approached when the total computation equals the total copy time, so a contiguous block can cost almost twice the optimum when copies dominate.
4. **Measurements on a GPU** (Section 6). With real layer kernels of two model shapes and page-locked host memory

on a Tesla T4, evenly spread resident layers are up to 1.50× faster than the contiguous prefix chosen by model-order filling the balanced placement reaches the bound with two slots where the contiguous prefix needs at least five, and the model orders placements correctly in 98.6% of the comparisons.

The PCIe Gen3 path of the T4 sustains about 12.3 GB/s of page-locked copies, the same bandwidth class as the CXL expanders measured by others [4, 5]; we use it as a real instance of a host-mediated path. We did not measure a CXL device and make no claim about its latency or behaviour under contention. Section 7 discusses what the theory implies for faster paths and for CXL memory.

2. Background and related work

Weight offloading for inference. FlexGen formulates the placement of weights, activations and the key–value cache over GPU, CPU and disk as a linear program over the fractions of each tensor type kept on each device, applied to every layer, and overlaps I/O with computation [1]. ZeRO-Offload and ZeRO-Infinity move parameters and optimizer state between GPU, CPU and NVMe memory in training [2, 8], and DeepSpeed-Inference streams weights from CPU or NVMe memory for inference [9]. PipeSwitch pipelines the transfer of a model's layers with their execution when switching models on a GPU and groups layers to balance transfer and computation [10]; it loads every layer once per switch rather than streaming a fixed subset every iteration. Hugging Face Accelerate maps modules to devices in model order and moves offloaded weights to the GPU when a module runs [3]; llama.cpp offloads a chosen number of layers to the GPU and computes the others on the CPU [11]. PowerInfer [12], Fiddler [13] and LIA [14] split computation between CPU and GPU instead of moving weights, and LLM in a flash reduces the bytes read from flash [15]. LOIP spreads retained and offloaded layers evenly over the stages of an edge pipeline to overlap loading with computation [7]. Our results explain why even spreading works, when it is optimal, and what contiguous placement costs, for the single-GPU pipeline that all of these systems contain.

Host-attached and CXL memory. CXL Type-3 expanders appear to the host as memory without a local processor [16, 17]. A GPU cannot address them directly; weights reach GPU memory by a host-initiated copy from a page-locked region, and TraCT reports that copies from an unpinned CXL region go through a bounce buffer [4]. ITME prefetches model weights layer by layer from a CXL-hybrid memory and reports about 18 GB/s [5]. DAK lets the GPU read remote memory directly [18], a different path from the one modelled here. Characterisations of real CXL devices report device-dependent bandwidth and latencies of a few hundred nanoseconds [6, 19]. In our model a CXL expander and host DRAM differ only in the transfer time per layer, which is an assumption we did not test on CXL hardware.

Table 1: Notation.

L	units (layers) per iteration
R, r	resident set and its size
$m = L - r$	streamed units per iteration
k	GPU slots for streamed units ($k \le m$)
c_i, f_i	compute and copy time of unit i
c, f	the same for uniform units
X_j, Y_n	completion of transfer j, execution n
$P(R, k)$	steady-state iteration time
g_j	resident units between streamed u_j, u_{j+1}

Swapping in training. Training systems swap tensors along a known dataflow graph: vDNN [20], SwapAdvisor [21], AutoTM [22] and Capuchin [23]. They decide what to evict and when to prefetch over one pass of a graph; inference repeats the same layer sequence every iteration with read-only weights, which is what makes a steady-state analysis possible. Integrated prefetching and caching for a known reference string has been analysed for disks [24]; Belady's rule [25] decides what to evict but not where resident items should sit in a cyclic sequence served by a single channel.

Max-plus algebra. Synchronised discrete-event systems such as pipelines with buffers are linear in the max-plus algebra, and their asymptotic cycle time is the maximum circuit mean of the associated event graph [26], computable for instance with Karp's algorithm [27]; see also [28]. Balanced (Sturmian) sequences are known to be optimal for routing and admission in max-plus systems [29]; our Theorem 2 is a result of the same flavour for a pipeline with a bounded buffer. We apply this framework to the weight-streaming pipeline and exploit the structure of its circuits to obtain closed forms.

3. Model

Figure 1 shows the system and Table 1 the notation.

Units and iterations. An inference iteration (one prefill chunk or one decode step of a batch) executes L *units* in a fixed order $0, 1, \ldots, L-1$; a unit is a transformer layer, or any other block whose weights move as one piece, such as the output head. Iterations repeat. Unit i takes $c_i > 0$ to compute once its weights are in GPU memory and $f_i > 0$ to copy from host-attached memory to the GPU; c_i depends on the number of tokens in the iteration and $f_i = a + s_i/B$ on the unit's size s_i, the path bandwidth B and a per-copy cost a.

Resident set and slots. A set R of r units stays in GPU memory. The other $m = L - r$ units, $u_0 < u_1 < \cdots < u_{m-1}$, are *streamed* every iteration through k GPU slots used as a ring, $1 \le k \le m$. The condition $k \le m$ means that a transfer never waits for a computation more than one iteration back; a runtime with more slots than streamed units uses only m of them. We number streamed transfers globally: transfer $j \ge 0$ carries unit $\sigma(j) = u_{j \bmod m}$ of iteration $\lfloor j/m \rfloor$ into slot $j \bmod k$.

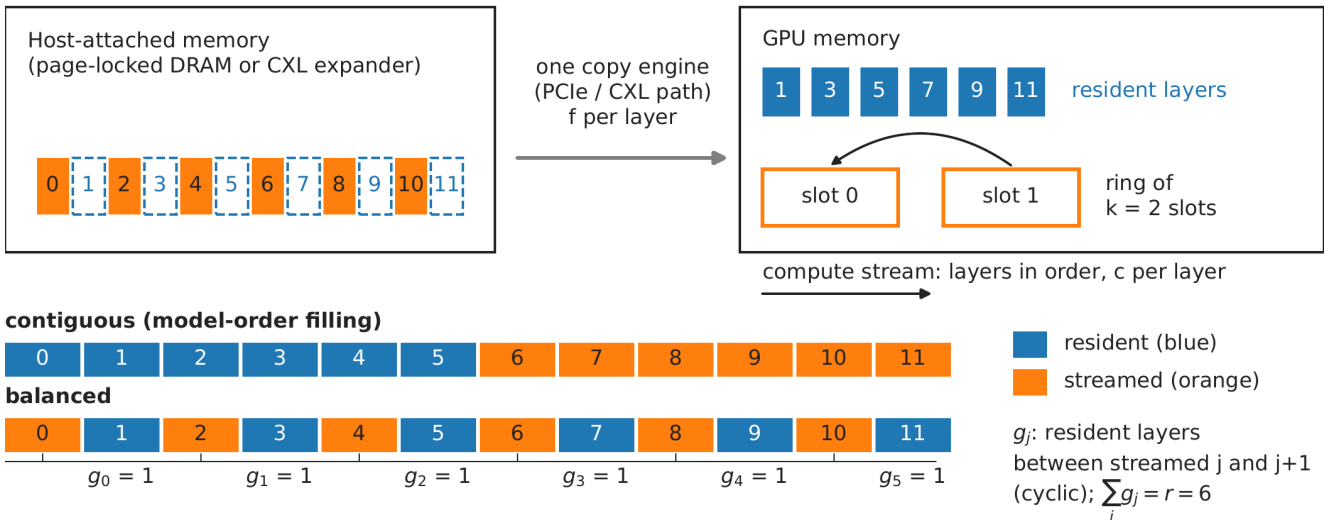


Figure 1: System model. Resident layers (blue) stay in GPU memory; the other layers (orange) are copied from host-attached memory into a ring of k slots by one copy engine while earlier layers compute. Bottom: the same 6 resident layers out of 12 placed as a contiguous block, as model-order filling does, and spread evenly (balanced), with the gaps g_j between consecutive streamed layers.

Assumptions. (A1) One copy engine performs the transfers one at a time in the order $j = 0, 1, \ldots$. (A2) The GPU computes units one at a time in program order. (A3) Transfer j may start as soon as the previous transfer has finished and its slot is free, that is, once the computation of the unit carried by transfer $j - k$ has finished (work-conserving prefetching). (A4) Times are deterministic and the same in every iteration. A1–A3 describe a runtime with one copy stream, one compute stream and events between them, which is how the runtime of Section 5 is built; A4 is relaxed in the measurements.

Recurrences. Let X_j be the completion time of transfer j and Y_n the completion time of the n-th unit execution (unit n mod L of iteration $\lfloor n/L \rfloor$), with $\nu(j)$ the execution index of transfer j's unit. Then

$$X_j = \max(X_{j-1},\, Y_{\nu(j-k)}) + f_{\sigma(j)}, \tag{1}$$

$$Y_n = \max(Y_{n-1},\, X_j) + c_{n \bmod L}, \tag{2}$$

where in (2) j is the transfer used by execution n; for a resident unit the second argument of the maximum is absent and $Y_n = Y_{n-1} + c_{n \bmod L}$. Terms of negative index are zero. These are max-plus linear recurrences: only maxima and additions occur.

Event graph and tilings. Draw one node per event X_j and Y_n and an arc from every event on the right-hand side of (1)–(2) to the event on the left, weighted with the time added (f for arcs into X, c for arcs into Y). Shifting all indices by one iteration (m transfers, L executions) maps the graph onto itself, so the system is periodic. A walk that starts at a node and returns to the same node shifted by $w \geq 1$ iterations is a *circuit spanning w iterations*; its weight is the sum of its arc weights.

Lemma 1 (Iteration time)**.** *Under A1–A4 the iteration time $P(R, k) = \lim_{n\to\infty} Y_{nL}/n$ exists and equals the maximum, over all circuits, of the circuit weight divided by the number of iterations it spans.*

Proof. Because $k \leq m$, every event of iteration n depends only on events of iterations n and $n - 1$. Collecting the events of iteration n into a vector $z(n)$ gives $z(n) = A_0 \otimes z(n) \oplus A_1 \otimes z(n-1)$ in max-plus notation. A_0 is acyclic, because within an iteration every arc goes from a lower to a higher index in the order $X_0, Y_{\nu(0)}, \ldots$ in which the events are generated by (1)–(2). Hence $z(n) = A_0^* A_1 \otimes z(n-1)$. Every node of the event graph has a path to the last execution of a later iteration (a transfer feeds an execution and every execution feeds the next), so every circuit lies upstream of Y_{nL}. For such a recurrence started from a finite initial vector, Y_{nL}/n converges to the maximum circuit mean of the precedence graph of $A_0^* A_1$ [26, 28], and circuits of that graph correspond to circuits of the event graph with the number of iterations spanned as the length. After a transient $Y_{nL} - nP$ may be periodic rather than constant, which is why the evaluator averages over many iterations. □

The arcs of the event graph are $Y_n \to Y_{n+1}$ (weight c_{n+1}), $Y_{\nu(j)} \to X_{j+k}$ (the slot arc, weight $f_{\sigma(j+k)}$), $X_j \to X_{j+1}$ (weight $f_{\sigma(j+1)}$) and $X_j \to Y_{\nu(j)}$ (weight $c_{\sigma(j)}$); all of them increase the iteration-relative position. Reading a circuit in this order gives the following description, which we use in all proofs. Executions are indexed modulo wL and transfers modulo wm, so segments may cross iteration boundaries.

Lemma 2 (Tilings)**.** *Every circuit spanning w iterations either uses only transfer nodes, in which case its weight is $w \sum_{i \notin R} f_i$, or it partitions the wL unit executions of w consecutive iterations into* singles*, each one execution n with weight $c_{n \bmod L}$, and* segments*. A segment begins right after the execution of a streamed unit carried by transfer j, ends with the execution of the unit carried by transfer $j + k + t$ for some $t \geq 0$, contains every execution in between, and has weight $\sum_{p=j+k}^{j+k+t} f_{\sigma(p)} + c_{\sigma(j+k+t)}$. A segment therefore contains $q = k + t \geq k$ streamed executions; it cannot begin or end inside a run of resident units. Conversely, every such partition is a circuit.*

Proof. A circuit that visits no execution node uses only arcs $X_j \to X_{j+1}$; it closes after wm transfers and has the stated weight. Otherwise, consider two consecutive visits of the circuit to execution nodes, Y_n and then $Y_{n'}$. Either the circuit takes the arc $Y_n \to Y_{n+1}$, so $n' = n+1$ and execution n' is a single; or execution n uses some transfer j and the circuit leaves by the slot arc to X_{j+k}, follows $t \geq 0$ arcs $X \to X$ to X_{j+k+t} (it cannot reach another execution node without passing $X \to Y$) and returns by $X_{j+k+t} \to Y_{\nu(j+k+t)}$, so $n' = \nu(j+k+t)$. In the second case the executions $n+1, \ldots, n'$ form a segment with the stated weight: the arcs used contribute the copy times of transfers $j+k, \ldots, j+k+t$ and the compute time of execution n', and the executions strictly between n and n' contribute nothing. Since the circuit returns to its first execution node after w iterations, these pieces partition the wL executions. Conversely, a partition into singles and segments defines, piece by piece, a sequence of arcs that closes after w iterations. □

A segment contains $k+t$ streamed executions and all resident executions between them; compared with covering the same executions by singles it changes the weight by its *gain*. For uniform units ($c_i = c$, $f_i = f$) and a segment that contains $q = k+t$ streamed units and G resident units,

$$\text{gain} = (t+1)f + c - c\,(q+G) = (q-k+1)(f-c) - (k-2)\,c - c\,G. \quad (3)$$

By Lemmas 1 and 2, the iteration time for uniform units is the larger of the transfer-only bound mf and Lc plus the largest gain per iteration achievable by a tiling, maximised over the number of iterations spanned.

Proposition 1 (Lower bound)**.** *For every resident set and every k, $P(R,k) \geq \max(\sum_i c_i,\ \sum_{i \notin R} f_i)$.*

Proof. The tiling made only of singles and the transfer-only circuit are circuits spanning one iteration. □

4. Placement theory

Throughout this section units are uniform: $c_i = c$ and $f_i = f$. This is the case of a dense transformer whose units are its layers; the output head and other irregular units are treated numerically in Section 4.1. We write $m = L - r$ for the number of streamed units and assume $2 \leq k \leq m$.

Gaps and balanced placements. For a resident set R let g_j be the number of resident units between streamed units u_j and u_{j+1} (indices modulo m, wrapping around the iteration), so that $\sum_j g_j = r$.

Definition 1. A placement is *balanced* if any two windows of the same length of the cyclic gap sequence $(g_0, \ldots, g_{m-1})$ have sums that differ by at most one.

In a balanced placement every window of q consecutive gaps sums to $\lfloor qr/m \rfloor$ or $\lceil qr/m \rceil$, since the average over all windows of length q is qr/m; this extends to windows longer than m, which contain whole turns of the cycle. Balanced placements exist for every r and m: the gaps $g_j = \lfloor (j+1)r/m \rfloor - \lfloor jr/m \rfloor$ form a balanced (Christoffel) word [30, Ch. 2]. In words, the resident layers are spread as evenly as the integers allow. The runtime of Section 5 places resident layers at positions $\lfloor (j+\frac{1}{2})L/r \rfloor$; the test suite checks that the resulting gap sequence is balanced for every $L \leq 130$ and every r, which covers all current dense models.

Theorem 1 (Per-layer compute bound)**.** *If $f \leq c$ and $k \geq 2$, then $P(R,k) = Lc$ for every resident set R.*

Proof. By (3), every segment has gain $(q-k+1)(f-c) - (k-2)c - cG \leq 0$. Hence every tiling has weight at most wLc, and the transfer-only circuit has weight $wmf \leq wLc$. Proposition 1 gives the reverse inequality. □

Theorem 1 says that resident layers are useless when a layer's computation hides its own copy: two slots (double buffering) already run at the speed of fully resident weights. The interesting case is $f > c$, in which copies are slower than computation per layer and resident layers must supply the missing computation time.

Theorem 2 (Balanced placement is optimal)**.** *If $f > c$, $k \geq 2$, $(k-1)f \geq 2c$ and the placement is balanced, then $P(R,k) = \max(Lc, mf)$, the lower bound of Proposition 1. In particular three slots suffice whenever $m \geq 3$, and two suffice when $f \geq 2c$.*

Proof. Consider a tiling spanning w iterations with segments $1, \ldots, N$, where segment ℓ contains q_ℓ streamed and G_ℓ resident units. The resident units of segment ℓ are those of q_ℓ consecutive gaps, so $G_\ell \geq q_\ell r/m - 1$ by balance. By (3),

$$\begin{aligned} \text{gain}_\ell &\leq (q_\ell - k + 1)(f-c) - (k-2)c - c\,(q_\ell r/m - 1) \\ &= q_\ell\,\frac{mf - Lc}{m} \;-\; ((k-1)f - 2c), \end{aligned}$$

using $L = m + r$. The second term is non-positive by assumption. Segments are disjoint, so $\sum_\ell q_\ell \leq wm$. If $mf \geq Lc$, the weight of the tiling is at most $wLc + \sum_\ell q_\ell(mf - Lc)/m \leq wLc + w(mf - Lc) = wmf$. If $mf < Lc$, every gain is negative and the weight is at most wLc. The transfer-only circuit has weight wmf. By Lemma 1, $P \leq \max(Lc, mf)$, and Proposition 1 gives equality. □

Theorem 3 (Contiguous placement)**.** *If $f > c$, $k \geq 2$ and the resident units form one contiguous block, then*

$$P(R,k) = \max(Lc,\ mf,\ rc + 2c + (m-k)f).$$

Proof. Lower bound. Besides the two circuits of Proposition 1, take the one-iteration tiling that covers the resident block and the first streamed unit after it by singles and the remaining $m-1$ streamed units by one segment with $t = m-1-k$; its weight is $rc + c + (m-k)f + c$. (If $m = k$ this term is at most Lc.)

Upper bound. Take a tiling spanning w iterations. Call an occurrence of the resident block *uncovered* if no segment contains it. A segment that contains the first streamed unit after an occurrence of the block also contains the block, so after an uncovered occurrence that unit is a single. The uncovered occurrences split the w iterations into stretches; consider a stretch of v iterations that contains $N \geq 1$ segments (a stretch without segments has

weight vLc). It has $v-1$ covered occurrences of the block and at most $vm-1$ streamed units in segments. Summing (3) over its segments, and using $(k-1)(f-c)+(k-2)c = (k-1)f-c > 0$,

$$\begin{aligned}\text{weight} &\le vLc + (f-c)(vm-1) - N((k-1)f-c) - c\,r\,(v-1)\\ &\le vLc + (f-c)(vm-1) - ((k-1)f-c) - c\,r\,(v-1)\\ &= v\,mf + (rc+2c-kf),\end{aligned}$$

so its weight per iteration is at most $mf + (rc+2c-kf)/v \le \max(mf,\ rc+2c+(m-k)f)$. If no occurrence is uncovered, all w block occurrences lie in segments, $\sum_\ell q_\ell \le wm$, and the same computation without the -1 terms gives at most $w\,mf$. Every circuit thus has mean at most $\max(Lc, mf, rc+2c+(m-k)f)$. □

The third term has a direct reading, which Figure 2 shows for $L = 12$, $r = 6$, $k = 2$, $c = 1$ and $f = 2$. While the resident block computes, the copy engine can load only the k streamed units that fit in the slots and then waits; the remaining $m-k$ copies happen while streamed units run at the copy rate, so the resident computation is not hidden behind copies. The contiguous placement takes $rc+2c+(m-k)f = 16$, the balanced one $\max(Lc, mf) = 12$, in which the copy engine never waits.

Corollary 1 (Cost of contiguous placement)**.** *Under the assumptions of Theorems 2 and 3, the contiguous placement is slower than the balanced one by the factor*

$$\frac{\max(Lc,\ mf,\ rc+2c+(m-k)f)}{\max(Lc,\ mf)} < 2 - \frac{c}{f}.$$

The bound is approached when $Lc = mf$ and $m \to \infty$ with k fixed.

Proof. Because $kf > 2c$, the numerator is less than $N = rc+mf$ unless it equals one of the first two terms, in which case the factor is 1. If $mf \ge Lc$, then $rc \le m(f-c)$ and $N/(mf) \le 2 - c/f$. If $Lc > mf$, then $N/(Lc) = 1+m(f-c)/(Lc) < 1+m(f-c)/(mf) = 2-c/f$. For the limit take $r = m(f/c-1)$, so that $Lc = mf$; the factor is $(rc+2c+(m-k)f)/(mf) \to 2-c/f$ as $m \to \infty$. □

The penalty therefore peaks when the total computation equals the total copy time, that is, when a balanced placement just hides every copy behind computation; a contiguous one then exposes the copies that cannot be prefetched during the resident block. For $f/c = 4.16$, the ratio of our shortest prefill, the bound is 1.76.

Proposition 2 (Two slots in the band $c < f < 2c$)**.** *If $c < f < 2c$, $k = 2$ and the placement is balanced, then* $\max(Lc, mf) \le P(R,2) \le \max(Lc, mf) + \frac{m}{2}(2c-f)$.

Proof. In the proof of Theorem 2 the second term of each gain is $2c-f > 0$, and every segment contains at least $k = 2$ streamed units, so there are at most $wm/2$ segments. □

In the band the bound is loose: in the random instances of the test suite the excess stays well below it, and for $L \le 12$ an exhaustive search over all placements finds no placement better than the balanced one (both checks are part of the test suite). Because a slot uses the same memory as a resident layer, two slots plus one more resident layer is often faster than three slots; the exact evaluator of Lemma 1 decides this in each case (Section 6.3).

Algorithm 1 Choosing slots and resident layers for a memory budget of M layers.

Require: per-layer times c_i, f_i measured in one all-resident iteration and one copy per layer; budget M
if all units have equal size **then**
  if $f \le c$ **then**
    $k \leftarrow 2$, $r \leftarrow 0$ ▷ Theorem 1: residents do not help
  else if $f \ge 2c$ **then**
    $k \leftarrow 2$, $r \leftarrow M-2$ ▷ Theorem 2
  else
    evaluate $k \in \{2,3\}$ with $r = M-k$ by Lemma 1; keep the faster ▷ Prop. 2
  end if
  $R \leftarrow \{\lfloor (j+\frac{1}{2})L/r \rfloor : 0 \le j < r\}$ ▷ balanced placement
else
  start from the uniform rule applied to the layers; keep large, rarely computed units resident if they fit
  improve R by local search (swap, add, remove) using the evaluator of Lemma 1
end if
return k, R, predicted $P(R,k)$

Guidelines. For a dense model with layers of equal size the analysis gives a complete recipe. (i) If a layer's computation is at least its copy time, use two slots and no resident layers beyond what is free. (ii) Otherwise use two slots if $f \ge 2c$ and three if $c < f < 2c$ (or evaluate both), fill the remaining memory with resident layers, and spread them evenly. (iii) The resulting iteration time is $\max(Lc, mf)$, so reaching compute speed requires $r \ge L(1-c/f)$ resident layers in addition to the slots. Algorithm 1 summarises the procedure, including the non-uniform case of Section 4.1.

4.1. Non-uniform units

Real models are not perfectly uniform: the output head of Llama-3 holds as many parameters as several layers but computes for only the last token in generation. Lemma 1 still gives the exact iteration time, and the recurrences (1)–(2) evaluate it by simulating a few hundred iterations, in time proportional to the number of unit executions. We use this evaluator inside a local search that swaps, adds and removes resident units subject to the memory budget (resident bytes plus k slots sized for the largest streamed unit) and compare it with the balanced placement of the layers alone (Section 6.4).

5. Measurement methodology

Platform. All measurements use a Tesla T4 (16 GB GDDR6, 40 streaming multiprocessors, PCIe Gen3 ×16) on a Google Colab virtual machine with 13.6 GB of host memory, PyTorch 2.14.1 and CUDA 13.0. Page-locked host-to-GPU copies of a layer (201 MB) run at 12.3 GB/s. The T4 has a 70 W power limit, and under sustained load its driver lowers the SM clock

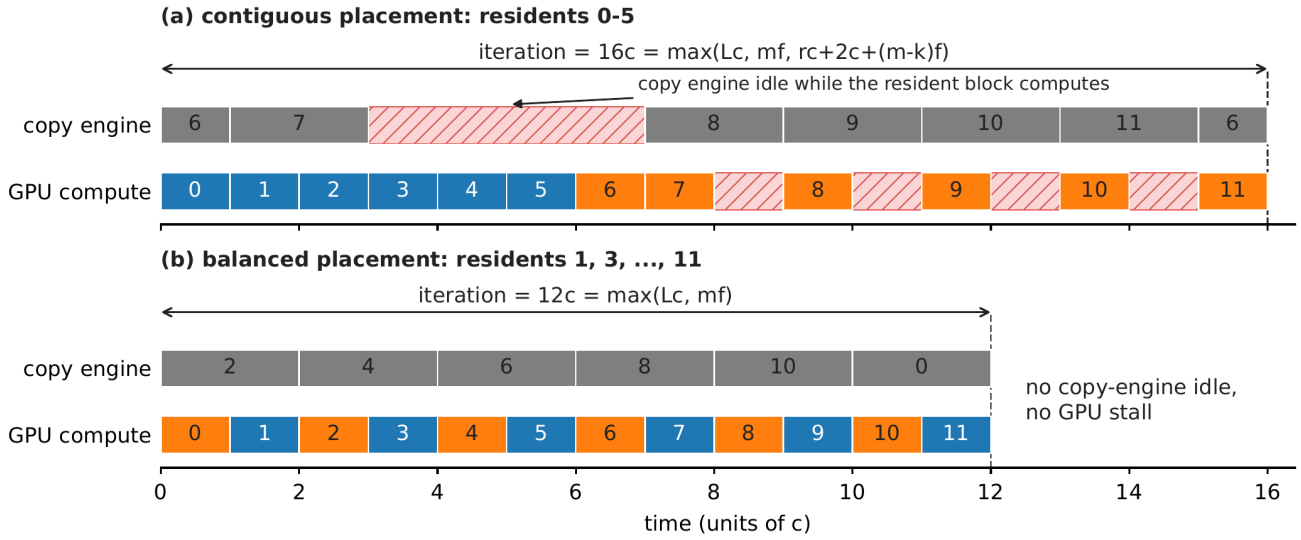


Figure 2: One steady-state iteration computed with the model for $L = 12$, $r = 6$, $k = 2$, $c = 1$, $f = 2$. (a) Contiguous placement: the copy engine is idle while the resident block computes, and the iteration takes 16. (b) Balanced placement: copies never stop, and the iteration takes the lower bound 12. Hatched: idle copy engine or GPU waiting for a copy.

to stay within it. In a separate diagnostic run (raw log in the repository) a concurrent copy, which itself draws power, slowed computation at default clocks by 5–9%, and the clock drifted down over tens of seconds of sustained load. Such clock changes make the compute time of a layer depend on the schedule, which assumption A4 excludes. The main measurements therefore lock the SM clock with `nvidia-smi -lgc` at a frequency at which the card stays below its power limit: 570 MHz for the Llama-3.2-3B prefill of 256 and 512 tokens and for its decode runs, and 450 MHz for all other runs, whose heavier work reached the power limit even at 570 MHz. Both values are about a third of the card's maximum clock of 1 590 MHz, so computation is two to three times slower than at default clocks, and the ratio f/c of every workload is correspondingly higher; the regime of a workload therefore depends on the clock as well as on its size, and the two locks are not comparable with each other. A sampler records the SM clock, power and throttle state every 200 ms, so every configuration can be checked for clock changes. Section 6.5 reports runs at default clocks and runs whose clock fell below the lock.

Model and workloads. We use the layer shapes of Llama-3.2-3B [31] (28 layers, model width 3 072, 24 query and 8 key–value heads of dimension 128, feed-forward width 8 192) with random FP16 weights. The amount of work does not depend on the weight values; their switching activity may affect power draw, which matters only where the clock is not locked. A unit is one transformer layer: RMS normalisation, the fused query–key–value projection, attention, the output projection, a second normalisation and the gated feed-forward block. Rotary embeddings are omitted. Attention uses the memory-efficient kernel of PyTorch with the key–value heads expanded to the query heads, because the T4 does not support the fused grouped-query path; the expansion adds a copy per layer and makes c somewhat larger than in a production kernel. Prefill iterations process one prompt of $T \in \{256, 512, 768, 1024, 1536, 2048\}$ tokens with causal attention; decode iterations process one new token for each of $B \in \{16, 64\}$ sequences against a key–value cache of 512 tokens that stays in GPU memory. One cache tensor is shared by all layers and is only read, not appended to; this keeps GPU memory free for resident layers and leaves the arithmetic per layer unchanged, but it may improve cache locality slightly compared with separate per-layer caches. With these shapes and clocks the ratio f/c of copy to compute time per layer ranges from 0.46 to 5.65, which covers the regimes of Section 4.

Runtime. The runtime implements assumptions A1–A3 directly. Every layer's weights are one contiguous slice of a single page-locked host buffer. Resident layers are copied to GPU memory once. Streamed layers use k GPU slots of one layer each, a copy stream and a compute stream: copy j waits on an event recorded after the computation of the layer that used slot $j \bmod k$ before it, and the computation of a streamed layer waits on the event recorded after its copy. All work is enqueued asynchronously from Python, so the GPU, not the host, decides when each operation starts as long as the host stays ahead; the host enqueue time per iteration is recorded. In short decode iterations with many resident layers the host was not far ahead, and launch overhead, which the model does not contain, may add to the iteration time. For each configuration the runtime executes two untimed iterations and then 12 timed prefill iterations (40 for decode), discards the first 4 (10), and reports the mean iteration time over the remaining 8 (30). Within a run, repeated configurations agree to well under 1%; between machines the variation is larger (Section 6.1). One further iteration records the start and end of every computation and copy with CUDA events, which gives the timelines in Figure 5.

Configurations. For every Llama-3.2-3B workload we run eight resident counts ($r = 0, 4, 7, 10, 14, 18, 21, 24$ of 28 layers), two slot counts ($k = 2, 3$) and four placements: the first r layers resident (*contiguous-first*, the order in which model-order filling assigns layers), the last r layers (*contiguous-last*), the balanced placement of Section 4 (resident positions $\lfloor (j + \frac{1}{2})L/r \rfloor$), and one random placement. For the Llama-3.1-8B shapes (16 layers, to fit host memory) we use $r = 0, 4, 8, 12$ with prefill of 512 and 1 024 tokens and decode with $B = 16$; for the model with the output head as an extra unit, $r = 0, 7, 14, 21$ with prefill of 512 tokens and decode with $B = 16$. A slot sweep at $T = 768$ and $r = 14$ uses $k = 1, 2, 3, 4, 6$. In total the measurements cover 605 streaming configurations plus one all-resident run per workload.

Predictions. Before the streaming runs of a workload we measure, with all weights resident, the compute time c_i of every layer (median of 7 timed repetitions after warm-up), and once per model the copy time f_i of every layer. The prediction for a configuration is the iteration time given by Lemma 1 for these measured c_i and f_i; nothing is fitted to the streaming runs. We also measure every layer's compute time while copies run back to back on the copy stream (c'_i) and evaluate a variant of the model in which a layer computes at rate $1/c'_i$ while a copy is in progress and at rate $1/c_i$ otherwise. With locked clocks c'_i exceeds c_i by at most 3.6%, so the two predictions are close; we report the plain model as the primary prediction and the variant where it matters. The prediction scales the measured c_i of a workload by one factor so that their sum equals the iteration time measured with all layers resident on the same machine. This is a calibration of the total compute time, not of the pipeline: no streaming configuration is used. The predictions are thus explanations of the measured iterations from per-layer times measured on the same machine, not a priori forecasts for other hardware.

6. Results

6.1. Accuracy of the model

Figure 3 compares the predicted and measured iteration times of all 605 streaming configurations of the main runs (clocks locked, both model shapes, prefill and decode). The prediction uses the per-layer compute times scaled so that their sum equals the measured all-resident iteration (scale factors between 0.97 and 1.00), and the per-layer copy times; nothing is fitted to the streaming runs. The mean absolute error is 0.29% and the largest error 5.24%; without the scaling the mean error is 0.39%. The average includes configurations that are easy to predict, so we also give it by regime: 0.28% for prefill with $f > c$, where placement matters, 0.18% for prefill with $f \leq c$, where every placement runs at compute speed, and 0.42% for decode. For the Llama-3.1-8B layer shapes the mean error is 0.48%. Eleven configurations have errors above 2%. Five of them, including the largest (5.24%), are 8B prefill runs with 12 resident layers whose clock fell to 433–437 MHz despite the 450 MHz lock; the model uses compute times measured at 450 MHz. Most of the others are short decode iterations with many resident layers, which last less than about 120 ms and in which the host enqueue time is close to the iteration time; the model contains no host overhead. Two runs of the same 46 configurations in separate Colab sessions differed by 1.74% on average (at most 3.05%). The model is more precise than this variation between sessions only because its inputs are measured in the same session as the iterations it predicts. Over the 285 pairs of placements with the same workload, resident count and slots whose measured times differ by more than 2%, the model orders all but 4 correctly (98.6%).

6.2. Placement

Figure 4 shows, for each prefill length, the iteration time of the contiguous and balanced placements relative to the lower bound $\max(\sum c_i, \sum_{\text{streamed}} f_i)$ as the number of resident layers grows. The regimes of Section 4 are visible. For $T = 256$ and 512 (f/c of 4.16 and 2.32, so $f \geq 2c$) the balanced placement stays at the bound for every r and both slot counts, as Theorem 2 states, while the contiguous one leaves the bound as soon as the resident computation exceeds what k slots can prefetch, and the gap widens until the total computation matches the total copy time. The largest measured gap is 1.50× (prefill of 256 tokens, $r = 21$, $k = 2$); Theorem 3 predicts 1.50× for this configuration, and Corollary 1 bounds the gap at this ratio by 1.76. At $T = 768$ ($f/c = 1.28$, inside the band $c < f < 2c$) the balanced placement with three slots stays within 0.40% of the bound, but with two slots it exceeds the bound by up to 8.3% (at $r = 7$; the model predicts 8.0%), the excess that Proposition 2 allows. Contiguous-last behaves like contiguous-first, as the theory expects for any single block. Random placements usually come close to balanced ones but not always: the worst random placement (decode with 64 sequences, $r = 14$, $k = 2$) was 14% slower than the balanced one. When a layer computes at least as long as it copies ($T \geq 1024$ for Llama-3.2-3B; at $T = 1\,024$ the two times are equal within the measurement noise, so this workload is only a boundary case), every placement, including streaming every layer, is within 0.42% of the all-resident time, as Theorem 1 states. For the 8B shapes at $T = 1\,024$, $f/c = 1.06$ and placement matters again. In decode the copies dominate unless almost all layers are resident, and placement matters only in the few configurations in which rc approaches mf.

Figure 5 shows why. In the contiguous configuration the copy engine fills the slots while the resident block computes and then waits for most of the block; afterwards the streamed layers run at the copy rate. In the balanced configuration the copy engine is never idle and the iteration ends when the last copy has been consumed.

6.3. Number of slots

At $T = 768$ a layer copies in 1.28 times its compute time, inside the band $c < f < 2c$ of Proposition 2. Figure 6 shows the slot sweep for $r = 14$. One slot serialises copy and computation for both placements. The balanced placement reaches the lower bound with two slots already (358.3 ms): the excess that Proposition 2 allows in this band, and that occurs for fewer resident layers (Section 6.2), does not occur for $r = 14$. The contiguous

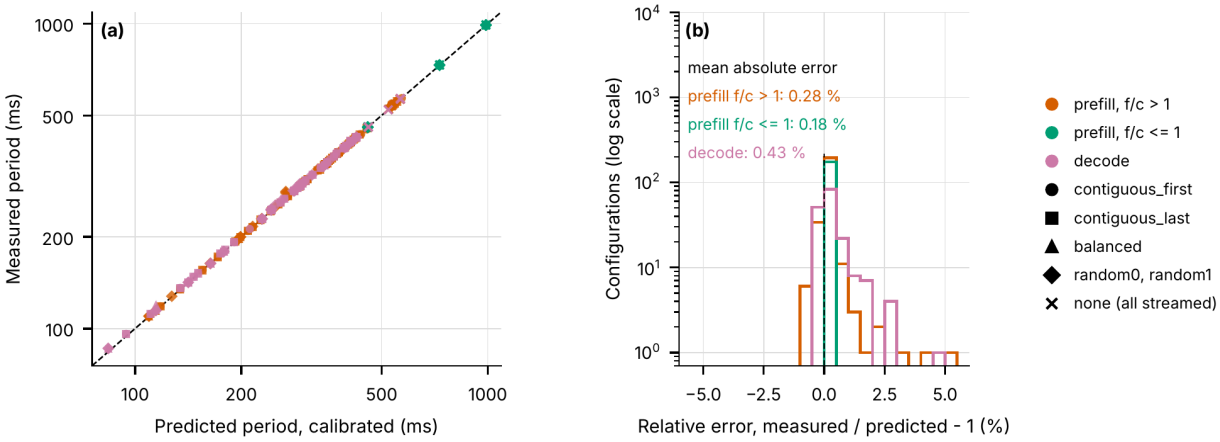


Figure 3: Accuracy of the model over all streaming configurations of the main runs (Tesla T4, locked clocks). (a) Measured versus predicted iteration time; colour: regime, marker: placement; the line is $y = x$. (b) Distribution of the relative error per regime.

placement improves with every additional slot, because each slot lets one more copy proceed during the resident block: 400.8 ms with two slots, 368.2 ms with four and 358.2 ms with six. By Theorem 3 it reaches the bound from $k = 5$ on (we did not measure $k = 5$). Since a slot occupies as much memory as a resident layer, the balanced placement reaches the same speed with the memory of at least three layers to spare. The model reproduces the sweep with a mean error of 0.16%.

6.4. Non-uniform units: the output head

With the Llama-3.2 output head as an extra unit (788 MB, four times a layer, but computed for one token only), the model predicts the measured iterations with a mean error of 0.41%. Figure 7 uses the calibrated per-unit times of this run in the model to compare placements under a memory budget; these comparisons are predictions, not measurements. Because the runtime sizes all slots for the largest streamed unit, streaming the head makes every slot as large as the head. Keeping the head resident and spreading the layers over the rest of the budget is then predicted to be 31.1% faster than the balanced layer placement with a streamed head at a budget of 15 layers with two slots (44.5% with three), and a local search over all placements improves on this rule by only 0.1%. A runtime with slots of different sizes would reduce the head's cost.

6.5. Clock changes and default clocks

To see whether the conclusions survive at default clocks, we repeated the Llama-3.2-3B prefill of 256, 512 and 1 024 tokens without a clock lock ($r = 0, 7, 14, 21$, $k = 2, 3, 78$ streaming configurations). The mean SM clock was 1 368, 1 114 and 680 MHz for the three lengths, so the GPU computed two to three times faster than in the locked runs and moved closer to its power limit as the work grew. The qualitative result holds: the balanced placement was never more than 1% slower than the contiguous one, and the contiguous placement was up to 1.21× slower (prefill of 256 tokens, $r = 21$, $k = 2$). The quantitative model holds much less well. Its mean error is 3.89% (maximum 19.50%), and it overestimates the cost of contiguous placement: for the configuration above it predicts 1.46 instead of the measured 1.21. Correcting the compute times by the measured mean clock of each run makes the error larger, not smaller (mean 5.65%), which suggests that the clock varies within an iteration rather than between configurations. A plausible explanation, which our 200 ms clock samples are too coarse to confirm, is that the GPU draws less power while it waits for copies, so its clock rises during the resident block that the model charges to contiguous placement. At default clocks on a power-limited GPU the model therefore ranks the placements correctly but overstates their difference.

In two earlier runs locked at 570 MHz, the heavier workloads reached the power limit and the clock fell below the lock during the streaming runs. With compute times measured before the clock fell, the model errs by up to 19% (mean 8.43%). Scaling each configuration's compute times by the ratio of the clock during the compute measurement to the clock during the run removes almost all of it (mean 0.63%, maximum 2.92%), so the error is the clock, not the pipeline model. In our very first runs, at default clocks and without clock sampling, the effect changed a conclusion: for a 1 024-token prefill with 14 resident layers the balanced placement measured 374 ms against 364 ms for the contiguous one, although the model predicts the opposite. Without clock samples we cannot confirm the cause. A plausible mechanism, consistent with the diagnostic run of Section 5, is that with a balanced placement copies overlap computation for the whole iteration, which raises power draw and lowers the clock while the computation is on the critical path.

7. Discussion

What runtimes should do. The rules of Section 4 cost nothing to apply. A runtime that streams layers with prefetching should spread its resident layers evenly instead of filling GPU memory in model order, and should size its ring by the ratio of copy to compute time per layer: two slots when a copy takes at least

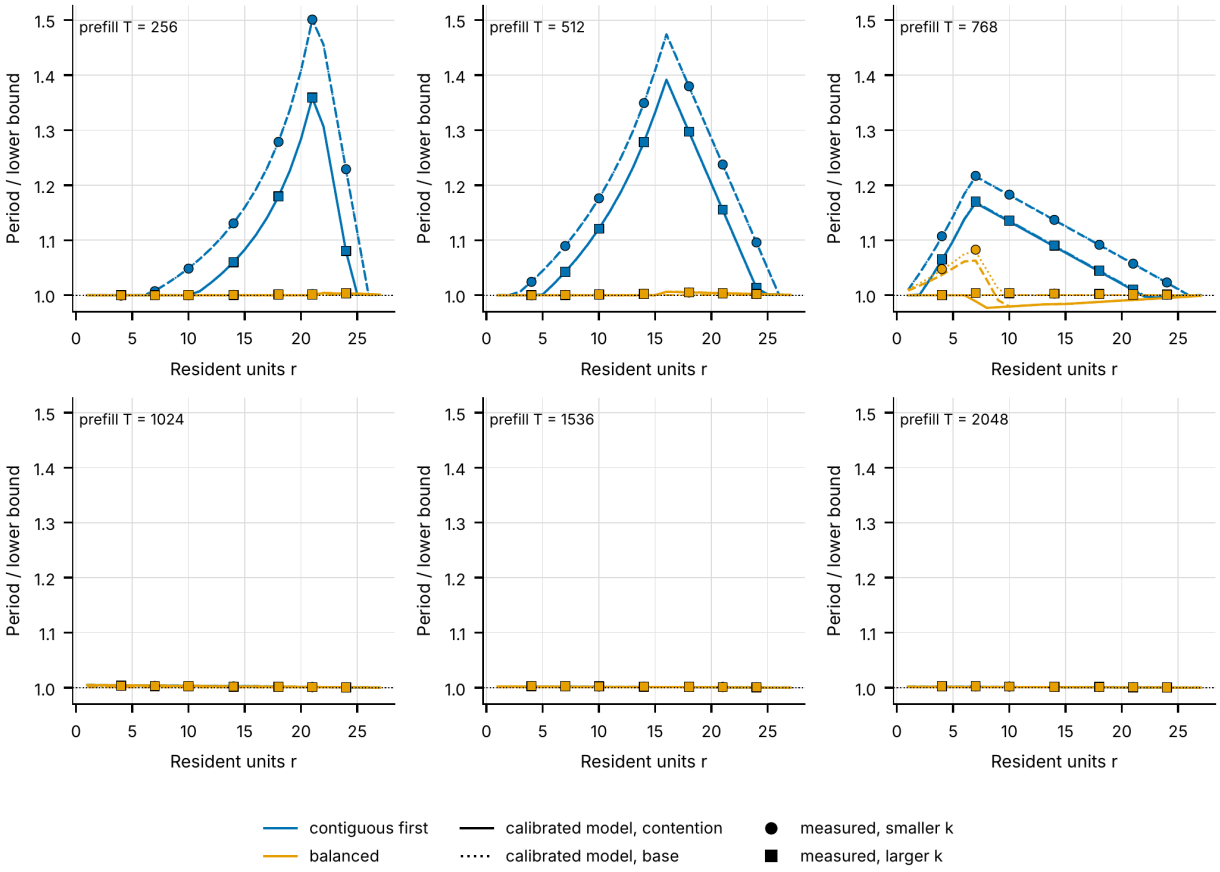


Figure 4: Iteration time relative to the lower bound versus the number of resident layers, Llama-3.2-3B prefill. Colours: contiguous-first and balanced placement. Circles and dotted or dashed lines: two slots; squares and solid or dash-dotted lines: three slots. Markers are measurements, lines the calibrated model (plain and with the contention variant, which nearly coincide).

twice as long as a layer's computation, three otherwise, or the faster of the two according to the evaluator, which needs only per-layer compute and copy times that a runtime can measure in one iteration. When a layer computes longer than it takes to copy, resident layers do not shorten the iteration at all, and the memory is better spent on a larger batch or key–value cache. When copies dominate even with every free byte resident, placement does not matter either, and only fewer bytes (quantisation) or a faster path help.

Faster paths and CXL memory. The analysis depends on the path only through the copy time per layer f. A CXL Type-3 expander reached through a page-locked host mapping is such a path [4, 5], and reported bandwidths for current devices are of the order we measured on PCIe Gen3. If copies from such a device take a constant time per layer, the theorems apply to it as stated. Whether they do is an open question that our measurements cannot answer: copies from an unpinned CXL region go through a bounce buffer [4], and contention on a shared device or with host memory traffic would make f vary from copy to copy. A faster path, such as several expanders interleaved or PCIe Gen5, lowers f and moves a given workload towards the regime $f \le c$ in which placement no longer matters; a GPU that computes faster moves it the other way.

Power-limited GPUs. On the power-limited T4 a concurrent copy and long runs lowered the SM clock, which changes c during an iteration and breaks assumption A4. The effect was large enough to reverse the order of two placements in one configuration of our first runs (Section 6.5). The contention variant of the model captures the part caused by concurrent copies but not the slower drift of the clock with temperature and power. Data-centre GPUs with higher power limits are less affected, but any deployment that runs close to its power limit should measure c under the same conditions as the iteration it wants to predict.

Limitations. (i) All measurements are on one GPU type, a power-limited T4 on Colab virtual machines, with two model shapes. The clock lock that makes the measurements reproducible runs the GPU at about a third of its maximum clock, which shifts every workload towards larger f/c; on a faster GPU or at default clocks the same workload has a smaller c and the regime boundaries move. (ii) The layer kernels omit rotary embeddings, expand grouped key–value heads explicitly, and share one read-only key–value cache across layers; each configuration is measured once, with one random placement per cell. (iii) The runtime is written in Python without CUDA graphs, and in the shortest decode iterations host launch overhead, which

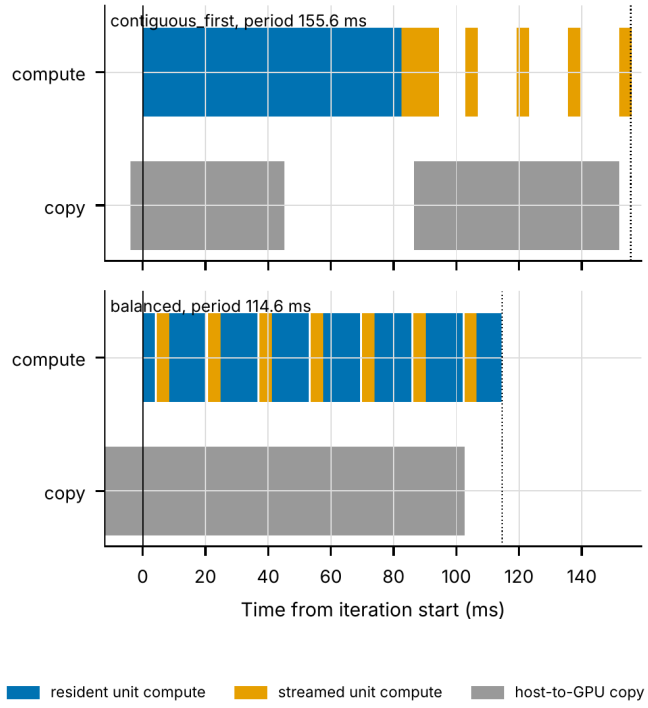


Figure 5: Measured timelines of one iteration (prefill of 256 tokens, $r = 21$, $k = 3$) for contiguous-first (top) and balanced (bottom) placement. Each panel shows computation (upper row) and copies (lower row).

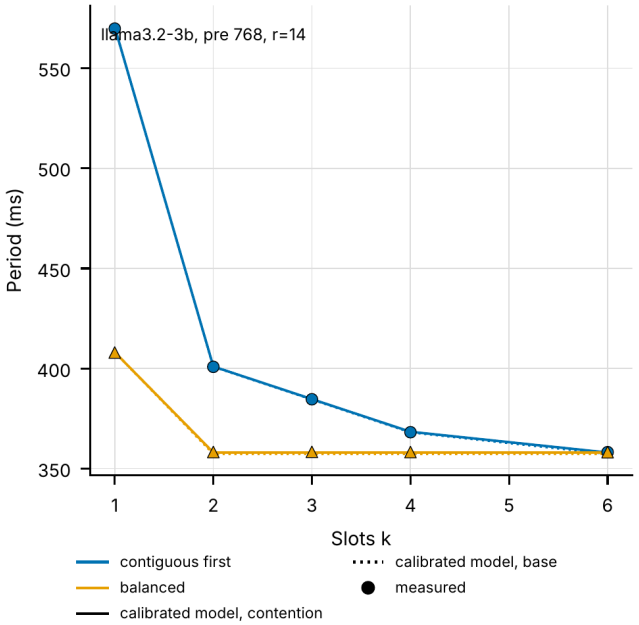


Figure 6: Iteration time versus the number of slots at $T = 768$, $r = 14$ (markers: measured; lines: model).

the model ignores, may be on the critical path. (iv) We stream whole layers; a runtime that splits layers into tensors has more, smaller units, to which Lemma 1 applies unchanged but for which we did not measure. (v) The key–value cache stays in GPU memory; streaming it as well adds per-layer copy time that grows with the context, which fits the model as a larger f_i but was not measured. (vi) We assume one copy engine and in-order execution; runtimes that reorder layers or split copies across several engines need a different event graph. (vii) The theorems cover layers of equal cost and require $k \le m$; for non-uniform units we offer an exact evaluator and a search, not a closed form.

8. Conclusion

Streaming the layers of a model through a small GPU-memory ring is a max-plus linear system, and its iteration time is the maximum mean of a family of tilings of the layer sequence. For layers of equal cost this gives a complete answer to the placement question. When a layer computes at least as long as its copy takes, two slots suffice and resident layers are useless. Otherwise, spreading the resident layers evenly attains the lower bound $\max(Lc, mf)$ as soon as $(k-1)f \ge 2c$, while a contiguous block, the result of filling GPU memory in model order, takes $\max(Lc, mf, rc + 2c + (m-k)f)$, up to $2 - c/f$ times as long. On a Tesla T4 with page-locked host memory, the same bandwidth class as current CXL expanders, the model, fed with per-layer times measured on the same GPU, reproduced 605 measured iteration times with a mean error of 0.29%, and even spreading was up to 1.50× faster than the contiguous prefix. The rules are cheap enough for any offloading runtime to adopt; the open questions are measurements on CXL devices and other GPUs, tensor-level units, streamed key–value caches and power-limited operation, in which the per-layer times themselves depend on the schedule.

Data availability

The model code, the GPU runtime, the raw measurements and the scripts that produce every number, table and figure are available in a public repository (link withheld for anonymous review).

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used generative AI-assisted tools for software implementation, experiment scripting and drafting and editing of text. After using these tools, the authors reviewed, verified and edited all code, results and content as needed and take full responsibility for the content of the published article.

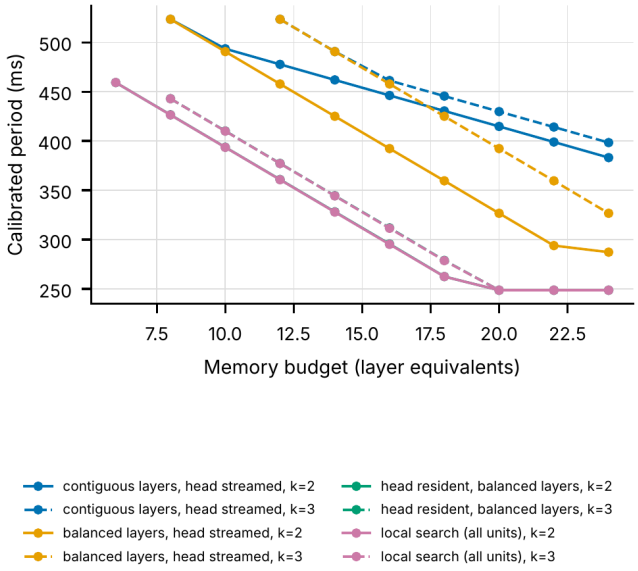


Figure 7: Predicted iteration time versus memory budget (in layer equivalents, including the slots) with the output head as a unit, Llama-3.2-3B prefill of 512 tokens, for two and three slots: contiguous layers, balanced layers, head resident plus balanced layers, and local search.

References

[1] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, C. Zhang, FlexGen: High-throughput generative inference of large language models with a single GPU, in: Proceedings of the 40th International Conference on Machine Learning (ICML '23), 2023.
URL https://proceedings.mlr.press/v202/sheng23a.html

[2] S. Rajbhandari, O. Ruwase, J. Rasley, S. Smith, Y. He, ZeRO-Infinity: Breaking the GPU memory wall for extreme scale deep learning, in: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC), 2021.

[3] Hugging Face, Accelerate documentation: Loading big models into memory, https://huggingface.co/docs/accelerate/en/concept_guides/big_model_inference, accessed 8 November 2026 (2026).

[4] D. Yoon, Y. Min, H. Kim, S. H. Noh, J. Kim, TraCT: Disaggregated LLM serving with CXL shared memory KV cache at rack-scale (2025). arXiv:2512.18194.

[5] H. Jang, Y. Min, S. Kim, T. Ahn, H. Kim, Y. Joo, H. Kim, J. Kim, ITME: Inference tiered memory expansion with disaggregated CXL-hybrid memories (2026). arXiv:2606.12556.

[6] Y. Sun, Y. Yuan, Z. Yu, R. Kuper, C. Song, J. Huang, H. Ji, S. Agarwal, J. Lou, I. Jeong, R. Wang, J. H. Ahn, T. Xu, N. S. Kim, Demystifying cxl memory with genuine cxl-ready systems and devices, in: Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO '23, ACM, New York, NY, USA, 2023, pp. 105–121. doi:10.1145/3613424.3614256.
URL https://doi.org/10.1145/3613424.3614256

[7] M. Sun, X. Zhang, S. Qu, Y. Li, M. Xiao, Y. Zheng, Y. Yuan, D. Yu, LOIP: Collaborative lossless LLM inference serving with offloading-based pipeline parallelism on edge devices (2025). arXiv:2512.21835.

[8] J. Ren, S. Rajbhandari, R. Y. Aminabadi, O. Ruwase, S. Yang, M. Zhang, D. Li, Y. He, ZeRO-Offload: Democratizing billion-scale model training, in: Proceedings of the 2021 USENIX Annual Technical Conference (USENIX ATC), 2021, pp. 551–564.

[9] R. Y. Aminabadi, S. Rajbhandari, A. A. Awan, C. Li, D. Li, E. Zheng, O. Ruwase, S. Smith, M. Zhang, J. Rasley, Y. He, DeepSpeed-Inference: Enabling efficient inference of transformer models at unprecedented scale, in: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC), 2022.

[10] Z. Bai, Z. Zhang, Y. Zhu, X. Jin, PipeSwitch: Fast pipelined context switching for deep learning applications, in: Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2020, pp. 499–514.

[11] G. Gerganov, contributors, llama.cpp: LLM inference in C/C++, https://github.com/ggml-org/llama.cpp (2023).

[12] Y. Song, Z. Mi, H. Xie, H. Chen, PowerInfer: Fast large language model serving with a consumer-grade GPU, in: Proceedings of the 30th Symposium on Operating Systems Principles (SOSP '24), ACM, New York, NY, USA, 2024, pp. 590–606. doi:10.1145/3694715.3695964.
URL https://arxiv.org/abs/2312.12456

[13] K. Kamahori, T. Tang, Y. Gu, K. Zhu, B. Kasikci, Fiddler: CPU-GPU orchestration for fast inference of mixture-of-experts models, in: Proc. Int. Conf. on Learning Representations (ICLR), 2025.

[14] H. Kim, N. Wang, Q. Xia, J. Huang, A. Yazdanbakhsh, N. S. Kim, LIA: A single-GPU LLM inference acceleration with cooperative AMX-enabled CPU–GPU computation and CXL offloading, in: Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA), 2025, pp. 544–558. doi:10.1145/3695053.3731092.

[15] K. Alizadeh, S. I. Mirzadeh, D. Belenko, S. K. Khatamifard, M. Cho, C. C. Del Mundo, M. Rastegari, M. Farajtabar, LLM in a flash: Efficient large language model inference with limited memory, in: Proc. 62nd Annual Meeting of the Association for Computational Linguistics (ACL), 2024, pp. 12562–12584.

[16] H. Li, D. S. Berger, L. Hsu, D. Ernst, P. Zardoshti, S. Novakovic, M. Shah, S. Rajadnya, S. Lee, I. Agarwal, M. D. Hill, M. Fontoura, R. Bianchini, Pond: Cxl-based memory pooling systems for cloud platforms, in: Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS '23, ACM, New York, NY, USA, 2023, pp. 574–587. doi:10.1145/3575693.3578835.
URL https://doi.org/10.1145/3575693.3578835

[17] H. A. Maruf, H. Wang, A. Dhanotia, J. Weiner, N. Agarwal, P. Bhattacharya, C. Petersen, M. Chowdhury, S. Kanaujia, P. Chauhan, Tpp: Transparent page placement for cxl-enabled tiered-memory, in: Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ASPLOS '23, ACM, New York, NY, USA, 2023, pp. 742–755. doi:10.1145/3582016.3582063.
URL https://doi.org/10.1145/3582016.3582063

[18] S. Lin, Z. Guo, J. Lin, DAK: Direct-access-enabled GPU memory offloading with optimal efficiency for LLM inference (2026). arXiv:2604.26074.

[19] D. Gouk, S. Lee, M. Kwon, M. Jung, Direct access, high-performance memory disaggregation with directcxl, in: 2022 USENIX Annual Technical Conference, USENIX ATC '22, USENIX Association, Berkeley, CA, USA, 2022, pp. 287–294.
URL https://www.usenix.org/conference/atc22/presentation/gouk

[20] M. Rhu, N. Gimelshein, J. Clemons, A. Zulfiqar, S. W. Keckler, vDNN: Virtualized deep neural networks for scalable, memory-efficient neural network design, in: Proc. 49th IEEE/ACM Int. Symp. on Microarchitecture (MICRO), 2016, pp. 1–13.

[21] C.-C. Huang, G. Jin, J. Li, SwapAdvisor: Pushing deep learning beyond the GPU memory limit via smart swapping, in: Proc. 25th Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2020, pp. 1341–1355.

[22] M. Hildebrand, J. Khan, S. Trika, J. Lowe-Power, V. Akella, AutoTM: Automatic tensor movement in heterogeneous memory systems using integer linear programming, in: Proc. 25th Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2020, pp. 875–890.

[23] X. Peng, X. Shi, H. Dai, H. Jin, W. Ma, Q. Xiong, F. Yang, X. Qian, Capuchin: Tensor-based GPU memory management for deep learning, in: Proc. 25th Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2020, pp. 891–905.

[24] P. Cao, E. W. Felten, A. R. Karlin, K. Li, A study of integrated prefetching and caching strategies, in: Proceedings of the 1995 ACM SIGMETRICS Joint International Conference on Measurement and Modeling of Computer Systems, 1995, pp. 188–197.

[25] L. A. Belady, A study of replacement algorithms for a virtual-storage computer, IBM Systems Journal 5 (2) (1966) 78–101.

[26] F. Baccelli, G. Cohen, G. J. Olsder, J.-P. Quadrat, Synchronization and

Linearity: An Algebra for Discrete Event Systems, Wiley, 1992.
[27] R. M. Karp, A characterization of the minimum cycle mean in a digraph, Discrete Mathematics 23 (3) (1978) 309–311.
[28] B. Heidergott, G. J. Olsder, J. van der Woude, Max Plus at Work: Modeling and Analysis of Synchronized Systems, Princeton University Press, 2006.
[29] E. Altman, B. Gaujal, A. Hordijk, Balanced sequences and optimal routing, Journal of the ACM 47 (4) (2000) 752–775.
[30] M. Lothaire, Algebraic Combinatorics on Words, Cambridge University Press, 2002.
[31] A. Dubey, et al., The llama 3 herd of models (2024). `arXiv:2407.21783`.